

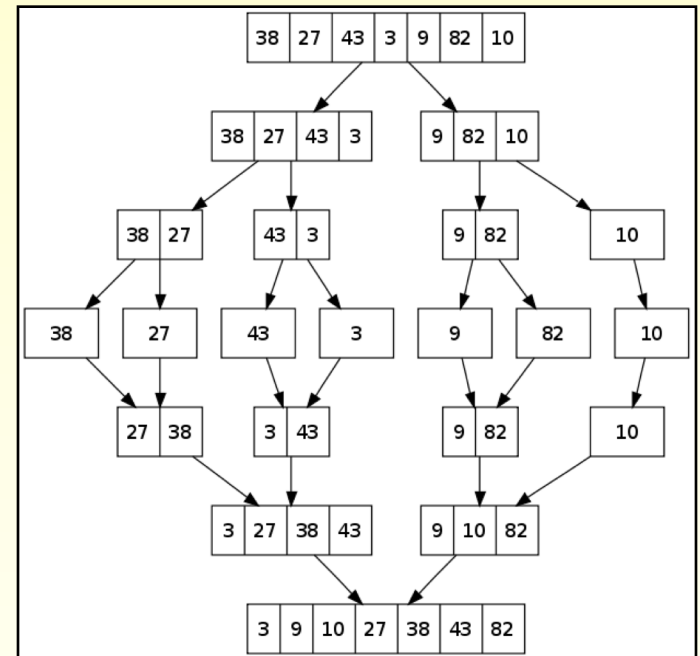
Merge-Sort Algorithm

Merge-Sort (*A*, *p*, *r*) { sort the elements in *A*[*p* ... *r*] }

1. if *p* < *r* then
2. $q \leftarrow \lfloor (p+r) / 2 \rfloor$
3. *Merge-Sort* (*A*, *p*, *q*)
4. *Merge-Sort* (*A*, *q*+1, *r*)
5. *Merge* (*A*, *p*, *q*, *r*)

$$\text{Work: } T_1(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ 2T_1\left(\frac{n}{2}\right) + \Theta(n), & \text{otherwise.} \end{cases}$$

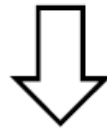
$$= \Theta(n \log n) \quad [\text{MT Case 2}]$$



Parallel merge-sort

Merge-Sort (A, p, r) { sort the elements in $A[p \dots r]$ }

1. *if* $p < r$ *then*
2. $q \leftarrow \lfloor (p + r) / 2 \rfloor$
3. *Merge-Sort* (A, p, q)
4. *Merge-Sort* ($A, q + 1, r$)
5. *Merge* (A, p, q, r)



Par-Merge-Sort (A, p, r) { sort the elements in $A[p \dots r]$ }

1. *if* $p < r$ *then*
2. $q \leftarrow \lfloor (p + r) / 2 \rfloor$
3. *spawn* *Merge-Sort* (A, p, q)
4. *Merge-Sort* ($A, q + 1, r$)
5. *sync*
6. *Merge* (A, p, q, r)